configuration for the `HttpClient` that will be injected into the typed client.

Listing 33.7 Registering a typed client with `HttpClientFactory` in Startup.cs

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
builder.Services.AddHttpClient<ExchangeRatesClient>                    ❶
        (HttpClient client) =>                                         ❷
    {                                                                  ❷
        client.BaseAddress =                                           ❷
            new Uri("https://example.com/rates/");                     ❷
        client.DefaultRequestHeaders.Add(                              ❷
            HeaderNames.UserAgent, "ExchangeRateViewer");              ❷
    })
    .ConfigureHttpClient((HttpClient client) => {});                   ❸
}

WebApplication app = builder.Build();
app.MapGet("/", async (ExchangeRatesClient ratesClient) =>
    await ratesClient.GetLatestRates());
app.Run();
```

❶ Registers a typed client using the generic AddHttpClient method

❷ You can provide an additional configuration function for the HttpClient that will be injected.

❸ As for named clients, you can provide multiple configuration methods.

Behind the scenes, the call to `AddHttpClient<ExchangeRatesClient>` does several things:

- Registers `HttpClient` as a transient service in DI. That means you can accept an `HttpClient` in the constructor of any service in your app and `IHttpClientFactory` will inject a default pooled instance, which has no additional configuration.